

高级 OOP：抽象类、多重继承、Mixin

前置：已掌握 class/self/继承、封装、@property、鸭子类型

目标：搞懂 Python 的抽象类怎么用、多重继承怎么避免踩坑

用时：约 15 分钟

相关笔记：[OOP深入（封装/魔法方法）](#) · [闭包与装饰器](#)

一、抽象类：Python 版的"接口"

先看问题

```
class Animal:
    def speak(self):
        pass # 子类自己去实现

class Dog(Animal):
    def speak(self):
        return "汪汪"

class Cat(Animal):
    def speak(self):
        return "喵喵"
```

这样写有个问题：没人强制子类必须实现 `speak()`。

```
class Fish(Animal):
    def swim(self): # 忘了写 speak()
        return "游啊游"

f = Fish()
print(f.speak()) # None——啥也没返回，也不报错，隐藏了 bug
```

用 ABC（Abstract Base Class）强制子类实现

```
from abc import ABC, abstractmethod

class Animal(ABC): # 继承 ABC
    @abstractmethod
```

```

def speak(self):                                # 加 @abstractmethod 装饰器
    """子类必须实现这个方法"""
    pass

class Dog(Animal):
    def speak(self):
        return "汪汪"

class Fish(Animal):
    def swim(self):          # 忘了实现 speak()
        return "游啊游"

# 关键区别：创建对象时就报错！
f = Fish()    # ✗ TypeError: Can't instantiate abstract class Fish
              #      with abstract method speak

```

ABC + @abstractmethod = "这个方法是接口定义，子类不实现就不让创建对象"。

抽象类的意义

```

from abc import ABC, abstractmethod

class Database(ABC):
    """定义了数据库操作的统一接口"""

    @abstractmethod
    def connect(self, url: str):
        pass

    @abstractmethod
    def query(self, sql: str):
        pass

    @abstractmethod
    def close(self):
        pass

class MySQL(Database):
    def connect(self, url):
        print(f"MySQL 连接: {url}")

    def query(self, sql):
        print(f"MySQL 查询: {sql}")

    def close(self):

```

```

    print("MySQL 关闭")

class SQLite(Database):
    def connect(self, url):
        print(f"SQLite 连接: {url}")

    def query(self, sql):
        print(f"SQLite 查询: {sql}")

    def close(self):
        print("SQLite 关闭")

def run_query(db: Database, sql: str):
    """不管什么数据库，只要继承 Database 就能传进来"""
    db.connect("localhost")
    db.query(sql)
    db.close()

run_query(MySQL(), "SELECT 1")    # ✓
run_query(SQLite(), "SELECT 2")  # ✓

```

抽象类 = 契约：告诉使用者"我保证有这些方法"，告诉实现者"你必须实现这些方法"。

ABC vs 鸭子类型

你可能觉得：鸭子类型不是"有这个方法就行"吗？那抽象类有什么用？

```

# 鸭子类型：灵活但不安全
def process_db(db):
    db.connect("...")
    db.query("...")    # 运行时才检查有没有这个方法
    db.close()

process_db("字符串")    # 运行时才报错：字符串没有 connect()

# 抽象类：安全但约束多
def process_db(db: Database):
    db.connect("...")
    db.query("...")    # 传入时就保证有
    db.close()

process_db("字符串")    # 类型检查就发现：字符串不是 Database

```

	鸭子类型	抽象类
灵活性	高——什么都能传	低——必须继承
安全性	运行时才暴露错误	创建对象时就能发现
适用场景	内部代码、小项目	公共 API、框架、团队协作

一句话记

ABC + @abstractmethod = 强制子类实现指定方法，不实现就不让创建对象。

二、多重继承：一个类有多个父类

Python 支持一个类继承多个父类——这是 Java 不支持的特性（Java 用接口替代）。

基本语法

```
class Flyer:
    def fly(self):
        return "飞行"

class Swimmer:
    def swim(self):
        return "游泳"

class Duck(Flyer, Swimmer):    # 继承两个父类
    pass

d = Duck()
print(d.fly())    # 飞行
print(d.swim())   # 游泳
```

一个 Duck 同时继承了 Flyer 和 Swimmer 的能力。

菱形继承问题（Diamond Problem）

```
class A:
    def greet(self):
        return "A"

class B(A):
    def greet(self):
```

```

        return "B"

class C(A):
    def greet(self):
        return "C"

class D(B, C):      # D 继承 B 和 C
    pass

d = D()
print(d.greet())    # 输出什么? B 还是 C 还是 A?

```

答案是 **B**。Python 用 **MRO (Method Resolution Order)** 决定调用顺序。

MRO：方法解析顺序

```

print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)

```

MRO 遵循 **C3 线性化算法**，规则是：

1. 子类优先于父类
2. 先继承的父类优先于后继承的父类
3. 保持单调性

所以找 `greet()` 时：

```

D 有没有? → 没有
B 有没有? → 有! 返回 B

```

super() 在多重继承中的行为

```

class A:
    def __init__(self):
        print("A.__init__")

class B(A):
    def __init__(self):
        print("B.__init__")
        super().__init__()

class C(A):
    def __init__(self):

```

```
        print("C.__init__")
        super().__init__()

class D(B, C):
    def __init__(self):
        print("D.__init__")
        super().__init__()

d = D()
```

猜输出顺序：

```
D.__init__
B.__init__
C.__init__
A.__init__
```

注意： `super()` 在多重继承里不是调父类，而是调 **MRO** 里的下一个类。

D 的 MRO： `D → B → C → A → object`

```
D.__init__ 调 super() → B
B.__init__ 调 super() → C（而不是 A！）
C.__init__ 调 super() → A
```

如果 B 和 C 的 `__init__` 没有调 `super()`，继承链就断了：

```
class B(A):
    def __init__(self):
        print("B.__init__")
        # 没调 super().__init__()

class C(A):
    def __init__(self):
        print("C.__init__")
        super().__init__()

class D(B, C):
    pass

d = D()
# B.__init__ → 结束，C 和 A 的初始化被跳过了！
```

规则： 在多重继承中，所有类都应该调 `super().__init__()`，否则继承链可能中断。

破坏实验

```
class A:
    def work(self):
        return "A"

class B(A):
    def work(self):
        return "B"

class C(A):
    def work(self):
        return "C"

class D(B, C):
    pass

class E(C, B):      # 和 D 的继承顺序相反
    pass

print(D().work())  # ?
print(E().work())  # ?
print(D.__mro__)   # ?
print(E.__mro__)   # ?
```

三、Mixin：多重继承的最佳实践

什么是 Mixin

Mixin = 专门给别的类"混入"额外功能的类。它不单独用，而是通过多重继承加到其他类上。

```
class JSONMixin:
    """给任何类添加 to_json() 方法"""
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class LogMixin:
    """给任何类添加 log() 方法"""
    def log(self, msg):
        print(f"[{self.__class__.__name__}] {msg}")

class User(JSONMixin, LogMixin):
```

```

def __init__(self, name, age):
    self.name = name
    self.age = age

user = User("张三", 25)
print(user.to_json())    # {"name": "张三", "age": 25} ← 来自 JSONMixin
user.log("登录成功")     # [User] 登录成功           ← 来自 LogMixin

```

Mixin 的命名规范

1. 名字以 Mixin 结尾
2. Mixin 类不单独实例化
3. Mixin 只提供方法，不定义 `__init__`

```

class TimestampMixin:
    """混入时间戳功能"""
    created_at: str # 类型标注，子类自己初始化

    def time_ago(self):
        return f"{self.created_at} 距今..."

class User(TimestampMixin):
    def __init__(self, name):
        self.name = name
        self.created_at = "2026-05-10"

u = User("张三")
print(u.time_ago())

```

Mixin 的实战例子

```

# 一个可打印的 Mixin
class PrintableMixin:
    def __str__(self):
        attrs = ", ".join(f"{k}={v}" for k, v in self.__dict__.items())
        return f"{self.__class__.__name__}({attrs})"

class Point(PrintableMixin):
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Student(PrintableMixin):
    def __init__(self, name, score):

```



```
        self.name = name
        self.score = score

p = Point(3, 4)
s = Student("张三", 92)
print(p)    # Point(x=3, y=4)
print(s)    # Student(name=张三, score=92)
```

一句话记

Mixin = 给你一个"只需继承、就能获得功能"的小工具类。名字以 Mixin 结尾，不单独用。

总结

概念	一句话	什么时候用
抽象类	ABC + @abstractmethod 强制子类实现方法	定义接口契约，公共 API
多重继承	一个类继承多个父类	需要组合多个独立功能
MRO	类名.__mro__ 查看方法查找顺序	搞不清调了哪个父类时查一下
super()	在多重继承中调 MRO 的下一个类	确保所有父类都被初始化
Mixin	通过多重继承给类"混入"额外功能	日志、JSON 序列化、打印等横切关注点

备考相关

- [EXAM_PREP/day04/00_今日任务](#) — Day 4 面向对象
- [EXAM_PREP/day06/00_今日任务](#) — Day 6 老师课堂代码重放